Adaptive Targeted Backtracking for Error Recovery in Sequential Multi-Agent Reasoning

Md. Fahmidul Hasan
2233129642
Department of Computer Science and Engineering
North South University
Dhaka, Bangladesh

Md. Fahim Fardin
2232842642
Department of Computer Science and Engineering
North South University
Dhaka, Bangladesh

***Abstract*—**Sequential multi-agent systems built with large language models pass intermediate reasoning from one agent to another, so an incorrect claim can influence later conclusions. This paper presents the Adaptive Targeted Backtracking Protocol (ATBP), a proposed approach for choosing among isolated claim repair, agent-level regeneration, dependency-aware branch backtracking, and full regeneration. The design represents intermediate reasoning as atomic claims, models prerequisite relationships as a dependency graph, and uses claim-verification signals to inform a recovery decision. The current prototype implements claim extraction, verification, and a full-regeneration baseline. In a preliminary evaluation of 10 GSM8K examples, the sequential baseline solved 4 examples correctly (40.0%); a separate no-fault test confirmed the verification-and-acceptance path but did not test recovery from an injected error. Therefore, adaptive recovery has not yet been experimentally validated. Planned comparisons will assess accuracy, repair success, error cascades, collateral changes to correct reasoning, token use, and latency.

***Index Terms*—**Large language models, multi-agent systems, reasoning verification, fault localization, error recovery, backtracking.

I. Introduction

In tasks like reasoning, question answering, planning, and problem solving, large language models (LLMs) have proven to be highly capable. Multi-agent systems have arisen as a means of distributing reasoning among several LLM-based agents as these jobs become more complicated. Several agents can produce, evaluate, validate, or improve intermediate outputs rather than using a single model to complete a task. Through a series of coordinated reasoning stages, this breakdown can offer specialization and enable complicated tasks to be tackled.

A sequential multi-agent system can be viewed conceptually as

$$A_1 \rightarrow A_2 \rightarrow A_3 \rightarrow \cdots \rightarrow A_n,$$

where each agent receives information produced by an earlier stage and uses that information in subsequent reasoning. Although such a structure can improve task decomposition and coordination, it also introduces an important reliability problem. An incorrect intermediate output may be passed to downstream agents, allowing an error introduced at one stage to influence subsequent reasoning. Recent studies have examined how multi-agent reasoning can benefit from step-wise error localization and iterative refinement, as well as how communication failures between agents can affect collaboration [1], [2].

If an intermediate error is not found right away, the task becomes more challenging. For instance, an early agent might make a false claim, a later agent might use that claim to draw a different conclusion, and later agents might accept the information as legitimate input. In this case, the final inaccurate response might not be the result of a failure at the end of the process, but rather the cumulative effects of an earlier mistake. As a result, fixing just the finished product might not deal with the root cause of the issue.

This leads to two connected reliability issues. The first is figuring out where the reasoning process went wrong. The second is figuring out how much of the thinking process should be recovered after the mistake has been found.

Regenerating the entire pipeline after identifying a mistake is a simple recovery method. This can eliminate the impact of a previous incorrect state, but it can also recreate previously correct logic, raising token and computing costs. On the other hand, if later agents have already assimilated the false information into their own reasoning, correcting just the specified claim can be effective but insufficient. As a result, the position of the fault and the degree to which subsequent reasoning depends on it may determine the proper recovery scope.

This leads to the study of adaptive recovery in sequential multi-agent reasoning. Instead of always using the same recovery method for every failure, a system could use details about the faulty claim, where it happened, and its downstream effects to choose the right recovery approach. Recovery could involve fixing a single claim, regenerating an affected agent, backtracking to an earlier step, or rebuilding the entire reasoning process.

This research, called the Adaptive Targeted Backtracking Protocol (ATBP), addresses the problem of recovery scope. The goal is not to present claim verification, fault localization, dependency modeling, or backtracking as new ideas in their own right. Instead, the study explores whether insights from these areas can help decide the right scope for recovery when intermediate faults occur in sequential multi-agent reasoning. The research focuses on balancing sound reasoning, correcting widespread errors, and avoiding unnecessary repetition.

The rest of this paper looks at earlier research on error propagation, fine-grained claim verification, fault localization, dependency-aware reasoning, backtracking, and adaptive recovery. These areas form the basis for the main question in this study: how to choose the scope of recovery when an intermediate error happens in a sequential multi-agent reasoning pipeline.

II. Literature Review

Research on reliable LLM-based systems has increasingly moved beyond evaluating only final outputs toward understanding how errors arise, propagate, and can be recovered within complex reasoning processes. For sequential multi-agent systems, this literature can be viewed through several connected research directions: error propagation, fine-grained claim verification, fault localization, dependency-aware reasoning, backtracking and rollback, and adaptive recovery. These directions provide complementary pieces of the reliability problem while also revealing the specific recovery-scope question investigated in this research.

A. Error Propagation in Multi-Agent Systems

In a sequential multi-agent workflow, one agent's output becomes context for a later agent. This creates a dependency between stages: a mistaken premise can be repeated, transformed, or treated as trusted evidence by later agents. Consequently, a final answer can be wrong even when the last agent follows the information it receives consistently. The relevant reliability question is therefore not only whether the final answer is correct, but also how intermediate information is transmitted and used.

Several published systems investigate how collaboration and role separation affect LLM outputs. Du et al. study multi-agent debate, in which independent model instances propose answers and critique one another over multiple rounds [3]. Chan et al. introduce ChatEval, a multi-agent debate framework for evaluating text quality, and show that role diversity can affect evaluation quality [4]. AgentBench evaluates LLMs across interactive agent environments and identifies weaknesses in long-horizon reasoning, decision-making, and instruction following [5]. These works establish the importance of coordination, feedback, and evaluation in agentic systems. However, they do not directly decide which portion of an already generated sequential reasoning pipeline should be invalidated after a particular intermediate claim is found to be wrong.

MAgICoRe combines coarse-to-fine multi-agent refinement and uses process-level signals to direct additional reasoning effort [1]. AgentAsk studies errors at communication boundaries and motivates asking for clarification when information exchanged between agents is uncertain [2]. Both are relevant to selective intervention: MAgICoRe directs refinement toward weaker reasoning, while AgentAsk intervenes at selected communication points. ATBP addresses a different decision after an error is detected: whether to repair a claim, regenerate an agent, recompute dependent downstream reasoning, or restart the process. That decision requires a representation of the fault's downstream effects, not only a signal that more discussion or clarification may be useful.

B. Claim-Level and Step-Level Verification

Before recovery can be chosen, the system needs a representation that is precise enough to distinguish a local error from a generally unreliable output. FActScore decomposes long-form generation into atomic factual units and assesses the support for each unit [6]. Its central lesson for ATBP is that a whole-response label can hide a mixture of valid and invalid components. MARCH brings proposition-level checking into a multi-agent hallucination-detection pipeline, using separate roles to solve, extract propositions, and check claims against evidence [7]. These studies support fine-grained verification, while ATBP uses the verification results as input to a subsequent recovery decision.

A growing body of peer-reviewed work evaluates reasoning at the level of individual steps. Sadeddine and Suchanek formulate verification of individual steps in deductive reasoning chains and investigate both language-model and neuro-symbolic approaches [8]. R-PRM uses explicit reasoning to improve process-level evaluation rather than relying solely on an opaque scalar score [9]. The Lessons of Developing Process Reward Models in Mathematical Reasoning examines annotation and evaluation issues in process reward models, including the need to consider both answer-level and step-level results [10]. Together, these studies are relevant to ATBP's claim verifier and to the evaluation of whether a detected fault was actually localized.

Other work explores robustness and granularity in process verification. RetrievalPRM retrieves related examples and reasoning steps to help a verifier judge unfamiliar problems and reasoning patterns [11]. FG-PRM addresses fine-grained hallucination detection and mitigation in mathematical reasoning and differentiates types of errors at the step level [12]. CodePRM uses execution feedback to score code-generation reasoning steps and employs a generate–verify–refine pipeline to identify and correct errors during code search [13]. CodePRM addresses a different task domain, but illustrates how external feedback can improve process verification. These studies also warn against treating a model-generated validity score as infallible: a verifier can miss an error, misclassify a valid step, or generalize poorly to unfamiliar inputs.

FIRE combines retrieval and iterative verification, allowing further checking when the available evidence is insufficient [14]. Its adaptive allocation of verification effort motivates a related question for recovery: should all detected faults trigger the same amount of regeneration? ATBP studies recovery scope rather than the number of verification rounds. The distinction matters because adaptive checking and adaptive recovery operate at different stages of a reliability pipeline.

C. Fault Localization

A verifier can identify a suspicious claim without necessarily determining where an error first entered the process or which subsequent conclusions rely on it. Fault localization therefore

connects intermediate evaluation with the choice of recovery action. RAFFLES investigates reasoning-based attribution of faults in LLM systems, including multi-agent and mathematical reasoning settings [15]. Its focus on attributing failure to specific stages complements the end-to-end accuracy metric used in many LLM evaluations.

Tyen et al. separately study the ability to find reasoning mistakes and the ability to correct a mistake when its location is given [16]. Their results show why these should be treated as distinct capabilities: successful correction with a known error location does not imply that a model can reliably find that location itself. Stepwise Verification and Remediation of Student Reasoning Errors similarly studies identifying a faulty step and generating feedback targeted to that step, although its application is educational tutoring rather than sequential multi-agent recovery [17]. These papers support explicit error localization and targeted feedback.

The remaining question is what to do after localization. Two faults can occur at the same stage but have different downstream consequences: one may affect a single dependent claim, while another may be embedded in the assumptions used by every subsequent agent. Thus, fault location alone may be insufficient for choosing recovery scope. ATBP uses the location together with dependency and cost information to formulate this decision, and the usefulness of those features must be demonstrated experimentally.

D. Dependency-Aware Reasoning

Reasoning processes can be represented as connected intermediate states rather than as one final answer. Graph of Thoughts represents thoughts as graph nodes and relationships between them as edges, enabling the combination and transformation of intermediate reasoning units [18]. For ATBP, the useful idea is not to reproduce the complete GoT search procedure, but to represent which claims depend on which prerequisite claims. Such a graph can distinguish a fault with a small downstream footprint from one that may invalidate a large portion of the reasoning state.

Language Agent Tree Search (LATS) combines language-model reasoning, action, and planning with Monte Carlo tree search, model-based value estimates, and reflection [19]. It demonstrates the usefulness of evaluating alternative branches and using feedback to guide search. Nevertheless, a search tree of candidate trajectories is not identical to a dependency graph over claims in a completed sequential pipeline. ATBP uses dependencies to estimate the impact of a known or suspected faulty claim and to determine which later reasoning units may need to be recomputed.

Dependency size should not automatically dictate the recovery action. A claim with many dependents may have been independently re-derived downstream, while a claim with only one dependent may encode a crucial assumption for the final answer. The graph therefore supplies decision features, not a guaranteed rule. Testing whether downstream dependency count, depth, upstream relationships, and estimated repair cost improve the recovery decision is part of the ATBP evaluation objective.

E. Backtracking, Search, and Iterative Correction

Tree of Thoughts permits LLMs to explore multiple candidate reasoning paths and use evaluation and backtracking when a path appears unpromising [20]. LATS similarly incorporates tree search and feedback into language-agent planning [19]. These methods show that a reasoning process need not remain committed to its initial trajectory. However, they primarily use search or backtracking to construct a better trajectory.

More directly, ReAgent presents a reversible multi-agent reasoning framework for multi-hop question answering and uses explicit local and global rollback when conflicts arise [21]. This is a close predecessor to ATBP and means that the contribution of ATBP should not be framed as introducing rollback or recovery boundaries. Instead, ATBP proposes to study whether claim-level verification, explicit claim-dependency features, and estimated recovery cost can guide a choice among four operational scopes in a sequential mathematical-reasoning pipeline. Because ATBP's adaptive policy has not yet been experimentally validated, this distinction remains a hypothesis to test.

Self-consistency improves reasoning by sampling multiple chain-of-thought paths and aggregating their answers [22]. It provides a useful reliability baseline based on answer agreement, but agreement between candidate answers does not identify the specific faulty claim or tell the system which downstream states should be recomputed. Reflexion uses feedback from earlier attempts to produce verbal reflections that guide subsequent trials [23]. It is relevant to learning from failures, although its feedback-and-memory mechanism does not, by itself, select a rollback boundary within a claim dependency graph.

Tyen et al. provide especially relevant evidence that giving a model the location of a reasoning error can make subsequent correction more effective [16]. The implication for ATBP is not that backtracking itself is new, but that localization and recovery should be considered together. ATBP extends this line of investigation by comparing four operational scopes—isolated claim repair, agent-level regeneration, dependency-aware branch backtracking, and full regeneration—under common task conditions.

F. Targeted Remediation and Adaptive Refinement

Several methods move beyond a one-shot answer and refine an output in response to feedback. Daheim et al. investigate stepwise verification and remediation of student reasoning errors, using identified errors to produce more targeted feedback [17]. MAgICoRe uses coarse-to-fine refinement to allocate additional reasoning effort where it is useful [1]. Reflexion stores verbal feedback from previous trials to guide later decisions [23]. These approaches share the general principle that correction should be informed by evidence about the prior attempt rather than applied blindly.

Process-verification methods also provide potential signals for such decisions. R-PRM evaluates intermediate reasoning

steps through explicit reasoning [9], while FG-PRM classifies fine-grained error types and uses step-level information to guide selection among generated solutions [12]. RetrievalPRM investigates how verification can generalize across reasoning patterns and problem distributions [11]. These methods can inform how ATBP detects or characterizes a fault, but they do not alone establish which recovery scope is most effective for a given dependency structure.

The distinction between refinement and recovery is central. Refinement may revise an answer or reasoning trace without explicitly invalidating all downstream outputs derived from a known faulty claim. In a sequential pipeline, later agents may have already consumed the faulty information. ATBP therefore treats recovery as a scope-selection problem: it attempts to repair only what is necessary, but expands the recovery boundary when a fault's downstream effects make a local correction inadequate. Whether this strategy improves accuracy and cost must be assessed against fixed alternatives rather than assumed.

G. Synthesis and Research Gap

The literature provides several complementary capabilities for reliable LLM systems. Multi-agent debate and evaluation frameworks study collaboration and judgment [3]–[5]; atomic and step-level verification methods expose errors inside generated reasoning [6]–[13]; fault-localization studies examine where reasoning fails and whether supplying an error location improves correction [15]–[17]; graph-based reasoning and search methods represent or explore connected reasoning states [18]–[20]; and iterative feedback or refinement methods use prior failures to improve subsequent outputs [1], [22], [23].

This prior work means that ATBP should not claim that verification, backtracking, or adaptive correction has never been studied. Instead, the specific research question is whether claim-level fault information, dependency structure, and estimated recovery cost can be combined to select among distinct recovery scopes in a sequential multi-agent reasoning pipeline. Existing approaches provide important components, but their goals and operational settings differ: some locate faults, some evaluate steps, some search over candidate reasoning paths, and others refine outputs or agents. ATBP proposes to connect these pieces in an explicit comparison of isolated claim repair, agent-level regeneration, dependency-aware branch backtracking, and full regeneration.

The central trade-off is between under-recovery and over-recovery. A narrow intervention may be inexpensive but leave incorrect downstream reasoning intact; a broad intervention may remove more faulty state but also discard correct reasoning and consume more tokens and time. The planned evaluation therefore considers final-answer accuracy alongside repair success, error cascades, unnecessary alteration of correct units, token usage, and latency. Because the current experiments use only a small preliminary GSM8K subset and have not yet validated the adaptive policy, the proposed recovery-scope advantage remains a hypothesis to test rather than an established result.

III. Dataset

The experiments use the GSM8K benchmark as the primary task domain. GSM8K consists of grade-school mathematical word problems that require multi-step reasoning to obtain a numerical answer. The benchmark is appropriate for this study because intermediate arithmetic and reasoning steps can be represented as individual claims and can therefore be verified separately.

The multi-agent system processes each problem through a sequential reasoning pipeline. Agent A acts as the planner/reasoner and generates an intermediate reasoning plan. Agent B receives the output of Agent A and performs the next stage of reasoning or verification. Agent C produces the final answer from the preceding reasoning.

For the current preliminary evaluation, 10 GSM8K examples were used. This should be considered a small-scale experimental subset rather than a full evaluation of the GSM8K dataset. The purpose of this initial evaluation is to establish the baseline behavior and validate the verification and recovery infrastructure before conducting a larger comparison of recovery strategies.

IV. System Architecture

The system is organized as a sequential multi-agent reasoning pipeline. The baseline architecture consists of three LLM-based agents:

$$\text{Problem} \rightarrow A \rightarrow B \rightarrow C \rightarrow \hat{y}.$$

Agent A serves as the planner/reasoner and produces an intermediate reasoning plan. Agent B receives this plan and performs the subsequent reasoning stage. Agent C receives the preceding output and generates the final answer.

The proposed recovery architecture adds an intermediate verification layer after Agent A. The output of Agent A is first decomposed into atomic claims. Each claim is then verified and represented together with its dependencies. If all claims satisfy the verification criterion, the original reasoning can continue to downstream agents. If a claim fails verification, the recovery mechanism can determine an appropriate recovery scope.

The broader ATBP design considers four recovery scopes:

1) **Isolated Claim Repair:** repair only the faulty claim.
2) **Agent-Level Repair:** regenerate the affected agent.
3) **Branch Backtracking:** regenerate the faulty claim and its dependent downstream reasoning.
4) **Full Regeneration:** discard and regenerate the complete reasoning process.

The current implementation establishes the infrastructure needed to support these choices but experimentally evaluates the full-regeneration strategy first. The adaptive selection of recovery scope remains part of the subsequent experimental evaluation.

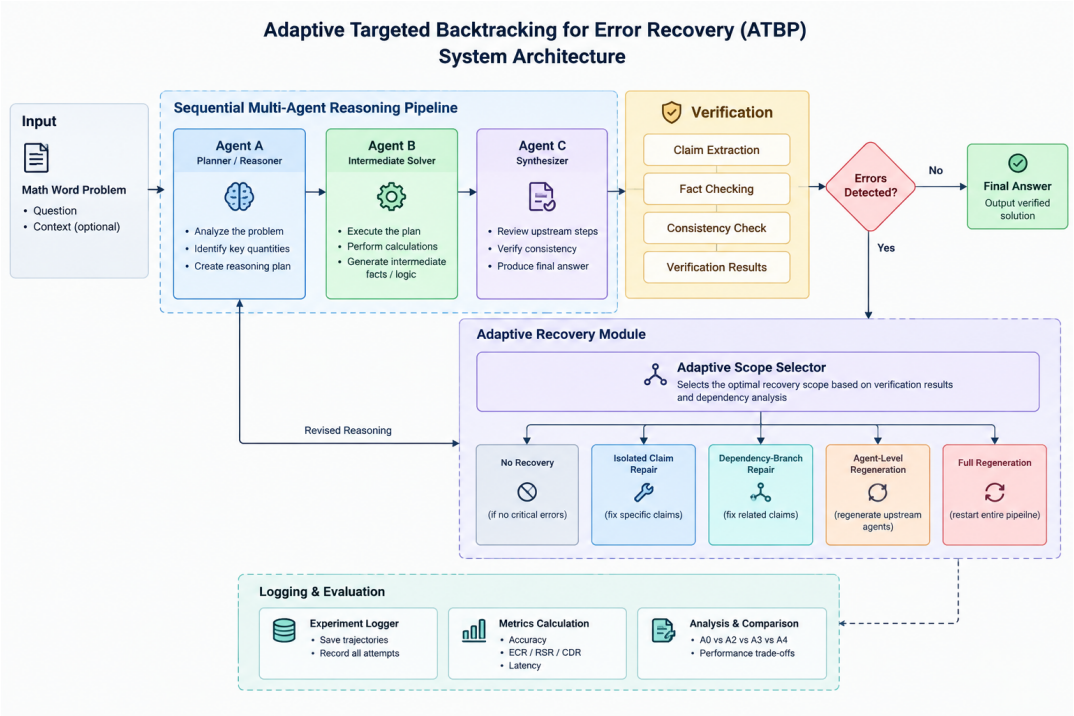


Fig. 1. Adaptive Multi-Agent Reasoning Architecture.

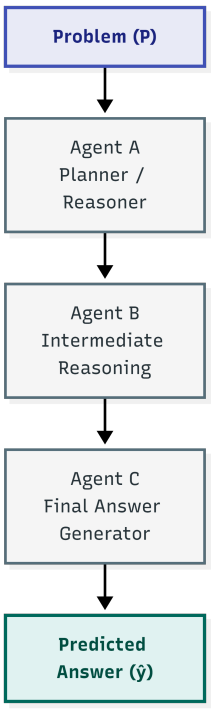


Fig. 2. Baseline sequential multi-agent reasoning architecture.

V. Methodology

The methodology consists of sequential multi-agent reasoning, atomic claim extraction, dependency modeling, claim verification, and recovery. The purpose of these components is to obtain enough fine-grained information about an intermediate reasoning process to support recovery decisions.

A. Sequential Multi-Agent Reasoning

Let the input problem be denoted by P. The baseline reasoning pipeline can be expressed as

$$P \rightarrow A \rightarrow B \rightarrow C \rightarrow \hat{y},$$

where A, B, and C represent the three sequential agents and $\hat{y}$ is the final predicted answer. The final-answer accuracy is defined as

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N},$$

where N_{correct} is the number of correctly solved examples and N is the total number of evaluated examples.

B. Atomic Claim Extraction

Agent A generates an intermediate reasoning sequence

$$R_A = \{s_1, s_2, \ldots, s_m\}.$$

Instead of treating the complete reasoning output as one indivisible object, the system transforms it into a set of atomic claims

$$C = \{c_1, c_2, \ldots, c_n\}.$$

Each claim is represented as

$$c_i = (id_i, text_i, D_i),$$

where id_i is the claim identifier, $text_i$ is the claim content, and D_i is the set of prerequisite claims on which the claim depends. The claim extractor is implemented using an LLM-based structured output prompt. The system requests a JSON representation containing a claim identifier, claim text, and dependency information. The resulting representation is normalized before verification. This decomposition is important because the recovery unit is smaller than an entire agent output. If a reasoning plan contains multiple claims and only one is invalid, claim-level representation allows the system to identify the potentially faulty component rather than automatically treating the complete output as faulty.

C. Dependency Graph

The extracted claims are represented as a directed dependency graph

$$G = (V, E),$$

where each vertex $v_i \in V$ corresponds to a claim and an edge $(c_i, c_j) \in E$ indicates that claim c_j depends on claim c_i. Thus,

$$(c_i, c_j) \in E \quad \Longleftrightarrow \quad c_i \in D_j.$$

The graph makes it possible to distinguish between a faulty claim with few downstream dependents and a faulty claim that influences a larger portion of the reasoning process. For a claim c, the downstream set is defined as

$$\mathrm{Down}(c) = \{v \in V \mid c \rightsquigarrow v\}.$$

Here, $c \rightsquigarrow v$ means that a directed path exists from claim c to claim v.

The dependency depth of a claim is defined recursively as

$$d(c) = \begin{cases} 0, & \text{if } Pred(c) = \emptyset, \\ 1 + \max_{p \in Pred(c)} d(p), & \text{otherwise.} \end{cases}$$

The current implementation includes graph construction, prerequisite and dependent relationships, upstream and downstream traversal, dependency-depth calculation, root and leaf identification, and cycle detection.

D. Claim Verification

Each extracted claim is passed to a claim-verification component. The verifier receives the original problem, the claim, and available reasoning context and returns two main judgments: whether the claim is valid and a confidence score. For a claim c_i, let

$$V(c_i) = (\mathrm{valid}_i, q_i),$$

where valid_i is the verifier's validity judgment and $q_i \in [0, 1]$ is its confidence. A verification threshold τ is used to determine whether a claim is accepted:

$$\mathrm{Accepted}(c_i) = \begin{cases} 1, & \mathrm{valid}_i = \mathrm{True} \wedge q_i \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

The current implementation uses

$$\tau = 0.80.$$

The verifier is currently implemented using the same local LLM-based infrastructure as the reasoning agents. Therefore, the verification confidence should be interpreted as a model-generated confidence estimate rather than as a calibrated probability. This limitation is important for later evaluation and motivates future comparison with more deterministic mathematical verification methods.

E. Verification with Full Regeneration

The first implemented recovery strategy is Verify + Full Regeneration, referred to as A2. After Agent A produces a reasoning plan, the system extracts and verifies its claims. If every claim is accepted, the plan is passed to the downstream agents. If at least one claim fails verification, the entire Agent A reasoning output is discarded and regenerated. Formally, if

$$\exists c_i \in C : \mathrm{Accepted}(c_i) = 0,$$

then the current reasoning state is rejected and a new Agent A output is generated. For regeneration attempt k,

$$R_A^{(k)} \rightarrow C^{(k)} \rightarrow V^{(k)}.$$

The generated reasoning is accepted when

$$\forall c_i \in C^{(k)}, \qquad \mathrm{Accepted}(c_i) = 1.$$

The current implementation allows up to three regeneration attempts after the initial generation. This strategy provides a controlled baseline for comparison with more targeted recovery methods because it does not attempt to select a smaller recovery scope.

F. Adaptive Recovery Scope

The broader ATBP protocol treats recovery as a selection problem. Let the set of possible recovery actions be

$$\mathcal{R} = \{R_{\mathrm{claim}}, R_{\mathrm{agent}}, R_{\mathrm{branch}}, R_{\mathrm{full}}\}.$$

These correspond to isolated claim repair, agent-level repair, dependency-aware branch backtracking, and full regeneration, respectively. For a detected faulty claim c_f, the recovery decision can conceptually depend on a feature vector

$$x_f = [q_f,\, d(c_f),\, |Down(c_f)|,\, |Up(c_f)|,\, cost_f],$$

where q_f represents verifier confidence, $d(c_f)$ represents dependency depth, $|Down(c_f)|$ measures the number of downstream dependents, $|Up(c_f)|$ represents upstream dependencies,

TABLE I
SYSTEMS EVALUATED.

System	Description
A0	Clean sequential multi-agent baseline (no verification).
A2	Verification followed by full regeneration of Agent A.
A3	Verification followed by fixed claim-level repair of every claim that did not pass verification.
A4	ATBP: verification and the dependency graph select one of four recovery scopes (isolated claim repair, dependency-branch repair, agent-level regeneration, full regeneration).

and $cost_f$ represents the estimated cost of recovery. The intended ATBP decision can therefore be expressed as

$$R^* = \pi(x_f),$$

where π is a recovery policy that maps fault characteristics to a recovery scope. At the current stage of the study, this adaptive policy is formulated as the central research objective but has not yet been experimentally validated. The present experiments instead establish the verification, dependency, and full-regeneration components required for evaluating the policy.

VI. Experimental Setup

The experiments use a local LLM inference environment through an OpenAI-compatible API. The reasoning agents, claim extractor, and claim verifier all use the same model, `llama3.2:latest`. Initial generation uses temperature zero. Regeneration (A2, and the agent-level and full-regeneration scopes of A4) uses temperature 0.7 with a different seed for each attempt, because a zero-temperature regeneration given the same prompt reproduces the rejected plan. The maximum generation length is 2048 tokens, the verification threshold is $\tau = 0.80$, and at most three regenerations are allowed per example.

Four systems were evaluated on identical examples (Table I). Every example is recorded for every system; a system that produces no answer for an example is counted as incorrect.

Metric computation.: ECR is computed over examples in which the initial verification rejected at least one claim (a detected intermediate fault); it is the share of those examples whose final answer is incorrect. This is an observed association, not a causal measure, because Agents B and C can also err. RSR counts each recovery action as one attempt: one per repaired claim for claim-level scopes, and one per regeneration attempt for regeneration scopes. An attempt succeeds when the affected claims are accepted on re-verification. CDR is the share of claims that were accepted before recovery, in examples where recovery ran, that recovery then discarded (regeneration) or rewrote (claim repair). A metric is reported as n/a when the evidence it needs is not available; for A0 no verification is run, so ECR, RSR and CDR are n/a. Token usage was not instrumented.

VII. Evaluation Metrics

The primary evaluation metric for the initial benchmark is final-answer accuracy:

TABLE II
FINAL-ANSWER ACCURACY ON 41 GSM8K EXAMPLES (95% WILSON INTERVAL). "NO ANSWER": EXAMPLES WHERE THE SYSTEM PRODUCED NO FINAL ANSWER; THESE COUNT AS INCORRECT.

System	Correct	Acc. (%)	95% CI	No answer
A0	26/41	63.4	[48.1, 76.4]	0
A2	27/41	65.9	[50.5, 78.4]	1
A3	26/41	63.4	[48.1, 76.4]	0
A4	27/41	65.9	[50.5, 78.4]	0

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N}.$$

For the later recovery experiments, additional metrics are required to measure both reliability and recovery efficiency. The Error Cascade Rate (ECR) is intended to measure the proportion of cases in which an intermediate fault propagates into downstream reasoning and contributes to an incorrect final result:

$$\text{ECR} = \frac{N_{\text{cascaded errors}}}{N_{\text{detected intermediate errors}}}.$$

The Repair Success Rate (RSR) is intended to measure how often a recovery action successfully resolves a detected intermediate fault:

$$\text{RSR} = \frac{N_{\text{successful repairs}}}{N_{\text{repair attempts}}}.$$

The Collateral Damage Rate (CDR) is intended to capture cases in which a recovery action unnecessarily changes or discards reasoning that was previously correct:

$$\text{CDR} = \frac{N_{\text{unnecessarily altered correct units}}}{N_{\text{correct units exposed to recovery}}}.$$

Computational efficiency will also be measured using token usage and latency. For a set of examples, average latency is

$$\overline{T} = \frac{1}{N}\sum_{i=1}^{N} T_i,$$

where T_i is the end-to-end runtime for example i. These additional metrics will only be reported after the corresponding instrumentation and experiments have been completed.

VIII. Results

All results are preliminary. They come from the first 41 GSM8K test problems (indices 0–40), on which all four systems were run to completion; the planned 100-example run was stopped early. With $N = 41$ the 95% confidence intervals are about ± 14 percentage points wide, so the results should be read as a pilot comparison rather than as evidence that any system is better.

TABLE III
RECOVERY METRICS IN % (NUMERATOR/DENOMINATOR). N/A: NOT COMPUTABLE BECAUSE THE SYSTEM PERFORMS NO VERIFICATION.

System	ECR	RSR	CDR
A0	n/a	n/a	n/a
A2	38.5 (5/13)	60.0 (12/20)	100.0 (90/90)
A3	33.3 (4/12)	70.6 (12/17)	0.0 (0/86)
A4	33.3 (4/12)	70.5 (31/44)	38.1 (32/84)

TABLE IV
MEAN END-TO-END LATENCY PER EXAMPLE (SECONDS). TOKEN USAGE WAS NOT INSTRUMENTED.

System	Mean latency (s)	Relative to A0
A0	35.4	1.00×
A2	86.2	2.44×
A3	62.0	1.75×
A4	66.0	1.86×

A. Final-Answer Accuracy

The sequential baseline A0 solved 26 of 41 examples (63.4%). A2 and A4 each solved 27 (65.9%), and A3 matched A0 at 26. The largest difference between any two systems is one example (2.4 percentage points), far smaller than the width of the confidence intervals. For 32 of the 41 examples the outcome was identical for all four systems (22 solved by every system, 10 by none); the systems differ on only 9 examples. A2 produced no answer on one example because all three regenerations failed verification.

B. Recovery Metrics

Error cascade rate. Verification detected at least one faulty claim in 12 to 13 examples per system, and between a third and 38.5% of those examples ended with an incorrect answer (Table III). The counts of examples with a detected fault differ slightly between systems (13 for A2, 12 for A3 and A4), which indicates that the initial Agent A plans were not identical across systems even at temperature zero. ECR differences of one example are within noise.

Repair success rate. A2 succeeded in 12 of 20 regeneration attempts (60.0%), A3 in 12 of 17 claim repairs (70.6%), and A4 in 31 of 44 recovery actions (70.5%). These values are not directly comparable across systems because the unit of an attempt differs (a regeneration of the whole plan for A2, a single claim for A3 and A4). Success is also judged by the same model that generated the reasoning, so it measures acceptance by the verifier and not independently confirmed correctness.

Collateral damage rate. CDR shows the expected ordering. A2 discarded all 90 previously valid claims that were exposed to recovery (100%), because full regeneration replaces the whole plan. A3 rewrote none of 86 (0%), since it repairs only the claims that failed. A4 rewrote 32 of 84 (38.1%), which are the valid claims that depend on a repaired claim in the branch-repair scope. A4 therefore sits between the two fixed strategies, consistent with its design of widening the recovery scope only when dependencies require it.

C. Computational Cost

Verification and recovery add cost relative to the baseline. A2 was the most expensive (2.44× A0), because regenerated plans are verified again. A4 matched A2's accuracy in this pilot at a mean latency 23% lower (66.0 s versus 86.2 s) while rewriting 38.1% rather than 100% of the exposed valid claims. Given the accuracy equivalence of all systems at this sample size, this is a descriptive observation about cost and collateral change, not a demonstrated advantage in accuracy.

D. Comparison with Published Results

Table V places our accuracy next to published GSM8K results. The comparison is indicative only: the published numbers use different models, prompting protocols and sample sizes, and none of the cited works evaluates claim-level verification with dependency-aware recovery scope selection. Our values are computed on 41 problems with wide confidence intervals.

Our zero-shot pipeline on a 3B-class model scores below Meta's reported 8-shot figure for the same model family and above its 1B figure. The gap to the 8-shot result is expected given the difference in prompting protocol, the three-agent pipeline, and the last-number answer extraction used here. The published results with larger models show a related pattern to ours: intrinsic self-correction by itself does not improve on standard prompting [24], whereas multi-agent debate can [3]. In our pilot, the recovery strategies did not change accuracy measurably (all paired $p = 1.0$), so we do not claim an accuracy gain.

Recovery metrics.: We did not find published work that reports ECR, RSR or CDR as defined here, so there is no external numeric baseline for these metrics. The closest related evidence is qualitative: Huang et al. [24] analyze how self-correction changes answers and find that it can turn correct answers into incorrect ones, which corresponds to the collateral change measured by CDR; and Tyen et al. [16] show that correction succeeds more often when the error location is supplied, which motivates the localized repair used in A3 and A4. Our recovery metrics are therefore compared across our own systems A0–A4 (Table III) rather than against other papers.

REFERENCES

[1] J. Chen, A. Prasad, S. Saha, E. Stengel-Eskin, and M. Bansal, "MAgICoRe: Multi-Agent, Iterative, Coarse-to-Fine Refinement for Reasoning," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Suzhou, China, 2025, pp. 32663–32686, doi: 10.18653/v1/2025.emnlp-main.1660.

[2] B. Lin, K. Yang, Z. Tan, Y. Lai, C. Zhang, G. Zhang, X. Yu, M. Yu, X. Wang, Y. Zhang, and Y. Wang, "AgentAsk: Multi-Agent Systems Need to Ask," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026, pp. 28055–28077, doi: 10.18653/v1/2026.acl-long.1294.

[3] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving Factuality and Reasoning in Language Models through Multiagent Debate," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235, 2024, pp. 11733–11763.

TABLE V

GSM8K ACCURACY: THIS WORK VERSUS PUBLISHED RESULTS. PROTOCOLS DIFFER ACROSS ROWS, SO DIFFERENCES SHOULD NOT BE READ AS RANKINGS. CI: 95% WILSON INTERVAL.

Source	Model	Method / protocol	Examples	Acc. (%)
Our work (zero-shot, three-agent pipeline)				
Our work	`llama3.2:latest`	A0: sequential baseline	41	63.4 [48.1, 76.4]
Our work	`llama3.2:latest`	A2: verify + full regeneration	41	65.9 [50.5, 78.4]
Our work	`llama3.2:latest`	A3: verify + claim-level repair	41	63.4 [48.1, 76.4]
Our work	`llama3.2:latest`	A4: ATBP (adaptive scope)	41	65.9 [50.5, 78.4]
Same model family, reported by the model developer				
Meta [**?**]	Llama 3.2 3B Instruct	8-shot chain of thought		77.7
Meta [**?**]	Llama 3.2 1B Instruct	8-shot chain of thought		44.4
Published self-correction and multi-agent debate (larger models)				
Huang et al. [24]	GPT-3.5	Standard prompting		75.9
Huang et al. [24]	GPT-3.5	Intrinsic self-correction, 2 rounds		74.7
Huang et al. [24]	GPT-3.5	Self-correction with oracle labels		84.3
Du et al. [3]	ChatGPT	Single agent		77.0
Du et al. [3]	ChatGPT	Reflection		75.0
Du et al. [3]	ChatGPT	Multi-agent debate		85.0
Huang et al. [24]	GPT-3.5	Debate vs. self-consistency (3 responses)		83.2 vs. 82.5

[4] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate," in *Proceedings of the International Conference on Learning Representations*, 2024.

[5] X. Liu et al., "AgentBench: Evaluating LLMs as Agents," in *Proceedings of the International Conference on Learning Representations*, 2024.

[6] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore, 2023, pp. 12076–12100,

[7] Z. Li, Y. Zhang, P. Cheng, J. Song, M. Zhou, H. Li, S. Hu, Y. Qin, E. Zhao, X. Jiang, and G. Jiang, "MARCH: Multi-Agent Reinforced Check for Hallucination," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026, pp. 39389–39415, doi: 10.18653/v1/2026.acl-long.1828.

[8] Z. Sadeddine and F. M. Suchanek, "Verifying the Steps of Deductive Reasoning Chains," in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 456–475,

[9] S. She, J. Liu, Y. Liu, J. Chen, X. Huang, and S. Huang, "R-PRM: Reasoning-Driven Process Reward Modeling," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Suzhou, China, 2025, pp. 13438–13451,

[10] Z. Zhang, C. Zheng, Y. Wu, B. Zhang, R. Lin, B. Yu, D. Liu, J. Zhou, and J. Lin, "The Lessons of Developing Process Reward Models in Mathematical Reasoning," in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 10495–10516,

[11] J. Zhu, C. Zheng, J. Lin, K. Du, Y. Wen, Y. Yu, J. Wang, and W. Zhang, "Retrieval-Augmented Process Reward Model for Generalizable Mathematical Reasoning," in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 8453–8468,

[12] R. Li, Z. Luo, and X. Du, "FG-PRM: Fine-grained Hallucination Detection and Mitigation in Language Model Mathematical Reasoning," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, Suzhou, China, 2025, pp. 4247–4278,

[13] Q. Li, X. Dai, X. Li, W. Zhang, Y. Wang, R. Tang, and Y. Yu, "CodePRM: Execution Feedback-enhanced Process Reward Model for Code Generation," in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 8169–8182,

[14] Z. Xie, R. Xing, Y. Wang, J. Geng, H. Iqbal, D. Sahnan, I. Gurevych, and P. Nakov, "FIRE: Fact-checking with Iterative Retrieval and Verification," in *Findings of the Association for Computational Linguistics: NAACL 2025*, Albuquerque, NM, USA, 2025, pp. 2901–2914, doi: 10.18653/v1/2025.findings-naacl.158.

[15] C. Zhu, S. Hong, J. Wu, K. Chawla, Y. Tang, Y. Yin, N. Wolfe, E. Babinsky, and D. Liu, "RAFFLES: Reasoning-based Attribution of Faults for LLM Systems," in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics*, 2026, pp. 7659–7688, doi: 10.18653/v1/2026.eacl-long.359.

[16] G. Tyen, H. Mansoor, V. Carbune, P. Chen, and T. Mak, "LLMs cannot find reasoning errors, but can correct them given the error location," in *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, 2024, pp. 13894–13908, doi: 10.18653/v1/2024.findings-acl.826.

[17] N. Daheim, J. Macina, M. Kapur, I. Gurevych, and M. Sachan, "Stepwise Verification and Remediation of Student Reasoning Errors with Large Language Model Tutors," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Miami, FL, USA, 2024, pp. 8386–8411, doi: 10.18653/v1/2024.emnlp-main.478.

[18] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of Thoughts: Solving Elaborate Problems with Large Language Models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17682–17690, doi: 10.1609/aaai.v38i16.29720.

[19] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235, 2024, pp. 62138–62160.

[20] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 11809–11822.

[21] X. Zhao, F. Gao, X. Song, Y. Chen, R. Yang, Y. Fu, Y. Wang, Y. Iwasawa, Y. Matsuo, and I. Li, "ReAgent: Reversible Multi-Agent Reasoning for Knowledge-Enhanced Multi-Hop QA," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Suzhou, China, 2025, pp. 4067–4089, doi: 10.18653/v1/2025.emnlp-main.202.

[22] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *Proceedings of the International Conference on Learning Representations*, 2023.

[23] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[24] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large Language Models Cannot Self-Correct Reasoning Yet," in *Proceedings of the International Conference on Learning Representations*, 2024.